

Verbatim Relay Fidelity of High-Entropy Credentials in LLM Tool Chains: An Empirical Study and Reference Architecture

Anonymous Authors

Paper ID: [To be assigned by EasyChair]

Double-anonymous submission to ICSA 2027 Research Papers Track

Abstract—Large Language Models (LLMs) increasingly orchestrate tool chains that invoke external services. When such a chain carries an opaque cryptographic credential, such as a JSON Web Token (JWT), the credential must remain byte-for-byte unchanged: a one-character deviation can cause downstream authentication to fail. We report a controlled experiment of credential relay through LLM-mediated Model Context Protocol (MCP) tool chains ($N = 48,000$ L2 runs across 16 frontier LLMs), an ecological validity arm that reproduces a production setting ($N = 1,600$), and an industrial case study. Across the controlled L2 runs, the JWT Defect/Failure Rate (DFR) was 18.0% (range 0.5–99.8%; 12.5% excluding the Mistral Large 3 outlier). Failures increased with credential length and were concentrated in the payload. In the production-condition arm, an explicit relay instruction left a residual DFR of 19.9% (1.9 percentage points above the neutral condition, $p=0.16$), providing no evidence that the tested prompt-based mitigation eliminates the failure mode. To address this integrity risk, we propose the Credential Interception Vault, a reference architecture that separates deterministic credential handling from probabilistic LLM orchestration, uses credentials by reference, and is model-agnostic. By removing credentials from the LLM inference path, the architecture eliminates direct byte relay by construction; we validate it in a production fintech deployment.

I. INTRODUCTION

The Model Context Protocol (MCP) [1] is increasingly used to connect Large Language Models (LLMs) with external tools and services [2], [3]. In a typical deployment, an LLM orchestrator selects tools and produces structured calls whose arguments may carry values from one tool invocation to the next. This arrangement is useful for multi-step workflows, but it can place an LLM on a path that was traditionally handled by deterministic software components.

One such path carries opaque cryptographic credentials, including JSON Web Tokens (JWTs) [4], [5]. These values are not instructions for the LLM to interpret; they are integrity-critical data that a downstream service must receive unchanged. A single altered character can invalidate cryptographic verification [6] and turn an otherwise valid tool invocation into a hard authentication failure. The resulting fault is operationally significant: the affected tool call fails, there is no meaningful near-miss credential, and format checks may not reveal every corrupted value before it reaches the downstream API.

This is an architectural problem rather than merely an instruction-following problem. Established credential-propagation patterns assume that the relaying components

preserve the credential value exactly. An LLM, however, is a probabilistic component that generates a new tool-call argument rather than acting as a transparent transport channel. Prompt instructions may encourage faithful relay, but they do not create a deterministic integrity guarantee. In systems where authentication, signing, or authorization depends on an exact value, placing the credential on this path introduces a failure mode that the architecture must address. In privacy-preserving architectures where authentication and signing occur locally and the resulting credential must be relayed through an LLM orchestrator to a remote API [7], this failure mode is directly exposed.

Existing work has examined unreliable LLM output, secret memorisation, and security risks in LLM-integrated systems [8]–[11]. These contributions are important but do not characterise the runtime integrity of a novel credential relayed through a live LLM tool chain, nor do they provide a reference architecture for removing that credential from the inference path. We therefore study the prevalence, character, and mitigability of JWT relay corruption, and evaluate an architectural response.

Our response is the Credential Interception Vault, a reference architecture that moves integrity-critical credential handling to a deterministic boundary. Rather than requiring the LLM to reproduce a credential, the Vault retains the credential outside the inference path and provides a reference that deterministic infrastructure resolves when constructing the downstream request. The design follows three principles: a deterministic shell around a probabilistic core, credentials by reference, and model-agnosticism. It preserves the LLM’s role in tool selection and orchestration while assigning byte-exact credential handling to components able to provide that property.

We advance the following research proposition:

For MCP deployments that would otherwise relay cryptographic credentials through LLM inference, prompt-engineering mitigations are insufficient to establish byte-exact relay integrity under current frontier models. A deterministic architectural bypass that removes the credential from the inference path eliminates this direct relay failure mode in the studied systems, independently of the LLM used.

We structure the investigation around five research questions:

- **RQ1:** How prevalent is JWT corruption in LLM-mediated

MCP relay, and how does it vary across the frontier model landscape?

- **RQ2:** What forms does JWT corruption take, and how are corruption events distributed across credential structure and length?
- **RQ3:** To what extent can prompt-engineering mitigations reduce JWT corruption, and why do they not establish an integrity guarantee?
- **RQ4:** How does the Credential Interception Vault reference architecture structurally eliminate the direct credential-relay failure mode?
- **RQ5:** How is the Vault operationalised in production, and what evidence demonstrates its effectiveness?

This paper makes seven contributions. The primary goal is to establish that the failure occurs at meaningful scale and that an architectural solution eliminates direct credential relay; causal attribution of the underlying mechanism remains a declared direction for future work (Section IX).

- 1) An empirical characterisation of JWT relay fidelity across 16 frontier LLMs in 48,000 controlled L2 runs, producing a cross-model defect rate ranking (**C1**).
- 2) A failure taxonomy distinguishing six outcome types with security severity ordering (**C2**).
- 3) A corruption characterisation identifying length-related and credential-segment concentration patterns, providing empirical evidence for examining the underlying mechanism (**C3**).
- 4) An evaluation of prompt-based mitigation showing that the tested explicit production instruction does not eliminate corruption (19.9% residual DFR under production instruction conditions) (**C4**).
- 5) The Credential Interception Vault, a model-agnostic reference architecture that removes credentials from LLM inference and thereby eliminates direct credential relay by construction (**C5**).
- 6) An industrial case study validating the architecture in a production blockchain trading system (**C6**).
- 7) An open replication package (**C7**).

Section II surveys related work; Section III presents the design; Section IV reports **RQ1–RQ3**. Sections V–VI present the Vault RA and case study (**RQ4–RQ5**). Sections VII–IX discuss, address threats, and conclude.

II. RELATED WORK

LLM Output Fidelity: LLM outputs are unreliable in ways that extend beyond factual hallucination [8], [9]. Nie et al. [10] study how code LLMs handle secrets at the token level, focusing on memorisation risk: the unintended reproduction of training-data credentials. No prior work studies the complementary failure: the *corruption* of novel, unseen credentials when an LLM relays them at inference time through a tool chain. This gap defines the empirical contribution of the present study.

Tokenisation and String Integrity: Byte Pair Encoding (BPE) [12] segments input into subword tokens using merge rules from a training corpus. Applied to high-entropy strings

such as JWTs, BPE produces a non-uniform distribution that Chen et al. [13] term *gibberish bias*: accidental natural-language fragments (e.g., “cut” inside Base64url) merge into single tokens while adjacent characters receive individual slots. Chen et al. measure KL divergence of 2.668 between credential-string and normal-code token distributions, and show the effect intensifies with vocabulary size (the current scaling trajectory). We hypothesise this same distributional shift underlies the relay fidelity failures we observe: rather than enabling memorised secret reproduction, it impedes accurate relay of novel credential strings. Chen et al.’s work concerns training-data secret *leakage*, not inference-time relay of novel credentials.

MCP and Agentic AI Security: The Model Context Protocol [1] enables LLMs to invoke external tools through structured function calls. Schick et al. [14] established that LLMs can learn tool use; He et al. [15] survey the resulting security and privacy risks across the broader LLM agent landscape. Abadeh et al. [11] provide the first peer-reviewed study of MCP-specific vulnerabilities, scanning server configurations for privilege escalation and injection risks. Their approach is complementary: static scanning examines infrastructure-level configuration; our study addresses a runtime failure mode that arises during live LLM inference and falls outside the scope of static analysis. Greshake et al. [16] and Liu et al. [17] study prompt injection attacks on LLM-integrated applications; the Vault architecture we propose incidentally mitigates indirect prompt injection targeting credentials, as the credential never enters the LLM context.

Function-Call and Tool-Use Reliability: Song et al. [18] find systematic argument misgeneration in LLM function-calling across diverse API schemas; Yu et al. [19] report compounding errors in multi-step tool-use, with both studies tolerating near-miss outputs that constitute a hard cryptographic failure in our setting. Haque et al. [20] study high-entropy string transcription failures in LLM code generation; that work does not address credential relay.

Credential Relay in Distributed Systems: The challenge of securely propagating bearer tokens across distributed components is well-documented in microservice architectures. Yarygina and Bagge [21] identify token propagation as a primary security challenge in microservices. These patterns exist for deterministic microservices, but they fail when a probabilistic LLM is introduced into the relay chain: the LLM is not a transparent relay component but an active transformer of its input. To our knowledge, no prior work addresses bearer token integrity when the relay component is a stochastic language model.

Reference Architectures for LLM Agent Systems: The Swiss Cheese Model for AI Safety [22], published at ICSA 2025, proposes a multi-layer guardrail reference architecture for foundation model agents, inspired by Reason’s accident causation model [23]. While conceptually adjacent, it addresses general agent failure modes (hallucination, misalignment, prompt injection) through stacked defensive layers. The failure mode we study is distinct: verbatim credential relay corruption

produces well-formed but cryptographically invalid output that passes format validation at every Swiss Cheese layer. Our architectural response is also distinct: rather than adding layers around LLM inference, the Credential Interception Vault removes the credential from the inference path entirely. The two architectures are complementary. Li et al.’s ACE architecture [24], published at NDSS 2026, isolates LLM plan generation from deterministic execution to prevent prompt injection. We apply the same separation principle to a different failure mode: verbatim string relay fidelity. In both cases, deterministic correctness guarantees cannot be provided by a probabilistic component, regardless of the operation’s apparent simplicity.

Grammar-constrained decoding [25] cannot guarantee byte-level fidelity of opaque strings: BPE tokenisation fragments the credential before generation begins, and no grammar can reconstruct a specific byte sequence already split across merge boundaries. OAuth 2.0 Token Exchange (RFC 8693 [26]) assumes a deterministic relay agent; the stochastic byte-level transformation we measure is outside its threat model.

The RA canon provides the classification framework for our contribution. Following Angelov et al. [27], we classify the Vault as a Type 2 facilitation-type RA: designed by researchers to guide practitioners, with voluntary adoption. Galster and Avgeriou [28] propose a six-step methodology for empirically-grounded RAs; our study satisfies all six steps (Section V-B).

III. STUDY DESIGN

The goal is to establish that JWT relay corruption is systematic (not model-specific), to quantify the limits of prompt-based mitigation, and to validate an architectural solution in production. A controlled experiment [29] measures prevalence and effect sizes; an industrial case study [30] provides contextual validation. The ecological validity arm bridges the two using production artifacts within the controlled framework.

The study follows Verdecchia and Bogner [31] and Kitchenham et al. [32]. Sections III-A–III-E describe the experiment; Section VI presents the case study design and findings together [30].

A. Experimental Objects

The experimental objects are synthetic JWTs generated per-run using `PyJWT` with a fresh HMAC-SHA256 secret and a randomised payload containing a UUID, Unix timestamp, user identifier, and nested JSON claims. Two design decisions motivate synthetic rather than real tokens. First, real JWTs from known systems risk appearing in LLM training data, introducing a contamination threat that synthetic tokens eliminate. Second, synthetic tokens allow cryptographic signature verification as an independent ground-truth check on corruption measurement.

JWTs are generated at five length tiers (Table I) to test the relationship between credential length and defect rate.

B. Experimental Subjects

Sixteen LLMs were selected across eight tiers (Table II) to provide broad coverage of the 2026 frontier landscape. The

TABLE I
JWT LENGTH TIERS.

Tier	Target	Rationale
Short	~200 chars	Minimal realistic JWT
Medium low	~300 chars	Below-typical; tests early-onset corruption
Medium	~500 chars	Typical production JWT
Long	~900 chars	Complex claims (e.g., role arrays)
Very long	~1,400 chars	Upper extreme; production RS256 token (~1,080 chars) falls in this tier

selection follows current guidelines for empirical SE studies involving LLMs [31], requiring commercial frontier models, open-source baselines, and longitudinal pairs (two Claude Sonnet generations, two Google Gemini generations) to test whether model-generation advancement reduces corruption. Chinese and European frontier models are included because different training corpora may produce tokenisers with different Base64url segmentation behaviour.

C. Variables

Independent variables. The experiment uses a mixed factorial design: (1) JWT length tier (5 levels: short, medium-low, medium, long, very-long); (2) context depth (3 levels: shallow ~100 tokens, medium ~1,500, deep ~3,000 tokens); (3) prompt ecological level (2 levels: L1, L2); and (4) temperature ($T = 0.0$ and $T = 0.7$). L2 was run at all factor combinations (100 iterations per cell); L1 at a single temperature and medium context depth to bound cost. The 15 API-hosted models each completed 3,000 L2 target runs; Gemma 4 E4B (Ollama local) completed 3,000 L2 runs, yielding 48,000 total L2 runs (100% of the 48,000 API-model target). L1 contributed 2,400 runs (16 models $\times$ 150 iterations across five JWT-length tiers, single temperature, medium context depth), bringing the combined main-arm total to 50,400. With the eco arm ($N = 1,600$; Section III-D), the overall study comprises 52,000 completed runs.

Dependent variables. The primary outcome is the *Defect/Failure Rate* (DFR): a binary indicator coded 1 if the output does not byte-for-byte match the input JWT. For corrupted runs, we compute the Levenshtein distance to measure severity and the Segment Corruption Rate (SCR) per JWT segment (header, payload, signature). Cryptographic signature verification provides an independent ground-truth check.

Outcomes are classified into six mutually exclusive categories (Table IV, Section IV-B). *Semantic substitution* is assigned when the output contains no dot-separated Base64url triplet but is non-empty (UUID extraction, JSON expansion, claim-value repetition). The replication package contains verbatim prompt templates, API model identifiers, SDK versions, tool schemas, request/response extraction, and error-handling logic.

D. Ecological Validity Arm

To test whether prompt-engineering mitigations can close the gap under production conditions, we conduct an ecological

TABLE II

MODEL TAXONOMY. ALL MODELS WERE API-VERSION PINNED AT THE START OF DATA COLLECTION. TIER CODES: WF=WESTERN FRONTIER, CE=COST-EFFICIENT, R=REASONING, CF=CHINESE FRONTIER, EF=EUROPEAN FRONTIER, LB=LEGACY BASELINE, SL=SONNET LONGITUDINAL, GL=GOOGLE LONGITUDINAL, LO=LOCAL/OPEN-WEIGHT.

Model	Tier	Provider	Model	Tier	Provider
Claude Sonnet 5	WF	Anthropic	Mistral Large 3	EF	Mistral
Gemini 3.5 Flash	WF	Google	Mistral Small 4	EF	Mistral
GPT-5.4-mini	CE	OpenAI	GPT-4o (pinned)	LB	OpenAI
Claude Haiku 4.5	CE	Anthropic	Claude Sonnet 4.5	SL	Anthropic
Gemini 3.1 Flash-Lite	CE	Google	Claude Sonnet 4.6	SL	Anthropic
o4-mini	R	OpenAI	Gemini 3 Flash	GL	Google
DeepSeek V3	CF	DeepSeek	Gemini 2.5 Flash	GL	Google
GLM-4 Flash	CF	Zhipu AI	Gemma 4 E4B	LO	Ollama

validity arm ($N = 1,600$; 16 models $\times$ 100 iterations) using the exact JWT schema and the verbatim 318-character BearerTokenParam tool instruction from the production codebase (Section VI), which names the RSA mechanism and warns that any single changed character produces a 403 error. A RESEARCH_RAW_MODE flag bypasses production mitigations. The pre-specified test is Spearman’s ρ between eco arm and main experiment model rankings.

E. Analysis Plan

Between-model inference uses Generalised Estimating Equations (GEE) with logistic link, binomial family, and exchangeable working correlation, clustered at the (model, cell) level. GEE estimates population-average effects and is robust to correlation misspecification via the sandwich variance estimator [32]. Per-model DFR estimates use Clopper-Pearson exact binomial 95% confidence intervals. Pairwise comparisons use Benjamini-Hochberg FDR correction at $\alpha = 0.05$. The eco arm uses Spearman’s ρ for rank correlation and Mann-Whitney U for the vocabulary effect test.

IV. EMPIRICAL STUDY RESULTS

A total of 50,400 runs completed (48,000 at L2, the primary analysis condition; 2,400 at L1). We report results organised by research question.

A. RQ1: Prevalence and Cross-Model Variation

The pre-declared primary aggregate is pooled DFR across all 16 models at L2 (48,000 runs); the 15-model figure excluding the Mistral Large 3 outlier is a pre-specified sensitivity check.

The overall JWT Defect/Failure Rate across 16 models under L2 conditions was **18.0%** (95% CI: [17.6%, 18.3%]). Approximately one in six authenticated tool-chain invocations produced a cryptographically invalid JWT.

Per-model DFR ranged from **0.5% to 99.8%** (Table III, Figure 1). No model achieved zero DFR.

Outlier analysis: Mistral Large 3. Mistral Large 3 (99.8% DFR, 3,000 L2 runs) is qualitatively distinct. Manual inspection of 50 outputs confirms explanation-mode semantic substitution as the dominant failure (99.0%): the model returns a description rather than the credential. Two causes are consistent: (i) a *tokenisation artifact* (aggressive BPE fragmentation favouring paraphrase), or (ii) a *model-configuration artifact* (the pinned version’s instruction-following profile prioritising explanation).

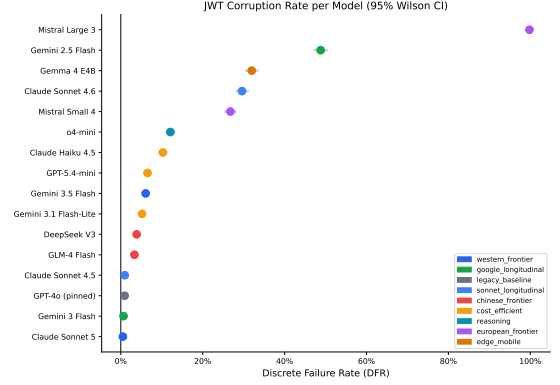


Fig. 1. Per-model JWT Defect/Failure Rate with Clopper-Pearson 95% confidence intervals, sorted ascending and grouped by tier.

The implication is that some models may be inherently unsuitable for direct credential relay, strengthening the case for an architectural bypass. Excluding this outlier, pooled DFR is **12.5%** (95% CI: [12.2%, 12.8%], $N = 45,000$, 15 models), confirming that the headline finding is not driven by a single outlier. BH-corrected pairwise Fisher’s exact tests across all $\binom{16}{2} = 120$ model pairs found 110 of 120 pairs (92%) significantly different at $\alpha = 0.05$, confirming near-universal heterogeneity.

No model exhibited safety-based refusal (0.0%; see taxonomy, Table IV). Cost-efficient models do not uniformly exhibit higher DFR than frontier models. The reasoning-tier model (o4-mini) shows a distinctive profile: chain-of-thought inference fragments credentials through an additional generation layer. Chinese frontier models show DFR consistent with different BPE vocabularies from Chinese-dominant corpora.

B. RQ2: Structural Anatomy of Corruption

Length-gating. DFR increases monotonically across tiers (Figure 2): short 12.6%, medium-low 12.6%, medium 13.6%, long 22.2%, very-long 28.7% (Spearman $\rho = 0.900$, $p = 0.037$). A Mantel-Haenszel test (high tiers: long + very-long vs. low tiers: short + medium-low; 16 model strata) confirms a significant length effect: OR = 4.785 (95% CI: [1.14, 20.0], $p < 0.001$). This result is identical when Mistral Large 3 is excluded (OR = 4.785, 15 strata, $p < 0.001$), confirming the length effect is not an artefact of that single outlier. Context depth DFR is monotonically increasing across three levels

TABLE III

PER-MODEL FAILURE-TYPE BREAKDOWN. VALUES ARE PERCENTAGES OF TOTAL L2 RUNS PER MODEL. DFR = 1— EXACT MATCH %. CLOPPER-PEARSON 95% CIs REPORTED FOR DFR ONLY.

Model	Tier	Exact %	Corrupt %	Sem.Sub %	Abst %	DFR %	95% CI
Mistral Large 3	EF	0.2	0.0	99.0	0.8	99.8	[99.6, 99.9]
Gemini 2.5 Flash	GL	51.2	48.7	0.0	0.1	48.8	[47.0, 50.6]
Claude Sonnet 4.6	SL	70.4	0.6	28.0	1.0	29.6	[28.0, 31.3]
Gemma 4 E4B	LO	68.0	28.9	1.4	1.7	32.0	[30.3, 33.7]
Mistral Small 4	EF	73.2	22.4	4.2	0.2	26.8	[25.2, 28.4]
o4-mini	R	87.9	5.4	0.0	6.7	12.1	[11.0, 13.3]
Claude Haiku 4.5	CE	89.7	10.0	0.3	0.0	10.3	[9.2, 11.4]
GPT-5.4-mini	CE	93.5	6.5	0.0	0.0	6.5	[5.7, 7.5]
Gemini 3.5 Flash	WF	93.9	5.2	0.0	0.9	6.1	[5.2, 7.0]
Gemini 3.1 Flash-Lite	CE	94.8	0.4	4.7	0.0	5.2	[4.4, 6.0]
DeepSeek V3	CF	96.1	3.8	0.1	0.0	3.9	[3.2, 4.6]
GLM-4 Flash	CF	96.7	1.8	0.0	1.6	3.3	[2.7, 4.0]
Claude Sonnet 5	WF	99.5	0.5	0.0	0.0	0.5	[0.3, 0.8]
Claude Sonnet 4.5	SL	99.0	1.0	0.0	0.0	1.0	[0.6, 1.4]
GPT-4o (pin.)	LB	99.1	0.5	0.5	0.0	0.9	[0.6, 1.3]
Gemini 3 Flash	GL	99.3	0.7	0.0	0.0	0.7	[0.4, 1.0]
All models (N=48,000)		82.0	8.5	8.6	0.8	18.0	[17.6, 18.3]

($\rho = 1.000$; $n = 3$ precludes a meaningful permutation p -value); A within-model GEE (groups = model, controlling for JWT length tier) yields OR = 1.093 (95% CI: [0.84, 1.42], $p = 0.51$), indicating no statistically significant independent context effect once model identity and token-length are held constant.

Segment analysis. Segment Corruption Rate (SCR) analysis reveals that payload corruption dominates: among character-level failures, 92.8% involve payload mismatch, followed by signature (11.3%) and header (0.4%). For the synthetic HS256 tokens used, the payload is the longest Base64url segment; this segment-level length-gating mirrors the JWT-level finding, consistent with BPE boundary exposure scaling with segment length. Among corrupted runs, 41.0% show single-character substitution (Levenshtein $L = 1$) and 35.1% show $L > 5$ (mean $L = 59.5$, pulled upward by near-total rewrites), indicating a bimodal severity distribution. DFR at fully deterministic sampling ($T = 0.0$) was 17.3%, only 1.3 pp below $T = 0.7$ (18.6%), confirming the dominant failure mode is deterministic, not sampling-noise-driven.

Failure taxonomy. Table IV presents the six-outcome taxonomy (C2). Character-level corruption (Levenshtein ≥ 1 , modal distance $L = 1$) is the dominant failure mode. Truncation (credential cut short) is a structurally distinct sub-mode observed in 0.4% of runs and reported merged with corrupted in Table III. Semantic substitution, where the LLM decodes the JWT payload and returns a plausible but cryptographically invalid value, is rare but architecturally critical: it is undetectable by any validation mechanism that does not hold a reference copy of the original token.

C. RQ3: Prompt Engineering as Insufficient Mitigation

The L1→L2 prompt transition produces a marginal, non-significant change in overall DFR (+1.1 pp, $p = 0.18$), confirming that increased instruction detail did not eliminate the observed corruption.

The ecological validity arm ($N = 1,600$) provides the key measurement under production conditions. Using the

TABLE IV

SIX-OUTCOME TAXONOMY OF JWT RELAY RESULTS (C2). THE ARCHITECTURAL IMPLICATION COLUMN MOTIVATES THE DESIGN DECISIONS IN SECTION V. REFUSAL HAD 0.0% OBSERVATION RATE IN THIS STUDY BUT IS ARCHITECTURALLY RELEVANT (SECTION V-A).

Outcome	Architectural Implication
exact_match	No intervention required
corrupted	Interceptor must hold reference copy
truncated	Credential shortened; same cryptographic impact as corrupted
sem_sub	Levenshtein detection insufficient; credential must not pass through LLM
abstention	LLM cannot be relied upon for credential forwarding
refusal	Out-of-band credential passing is the only solution

production BearerTokenParam instruction (318 characters, explicitly naming the RSA signature consequence), the DFR was **19.9%**. The difference from the main study at the same length tier was +1.9 percentage points ($p = 0.16$, Wilcoxon; not significant). Spearman’s rank correlation between eco arm and main experiment model rankings was $\rho = 0.740$ ($p = 0.001$), confirming that model rankings transfer to production conditions.

The most explicit instruction achievable within the MCP tool schema design space produces a result statistically indistinguishable from neutral vocabulary. Corruption persists regardless of prompt specificity; BPE tokenisation boundaries appear to operate below the level of instruction-following behaviour and were not overcome by any prompt tested in this study. We note that non-significance alone does not constitute formal equivalence: establishing a true prompt ceiling would require pre-specified equivalence margins and a TOST procedure, which is identified as future work (Section IX).

V. REFERENCE ARCHITECTURE

A. From Taxonomy to Architectural Requirements

The six-outcome taxonomy (Table IV) is not merely a measurement scheme; each outcome implies a specific archi-

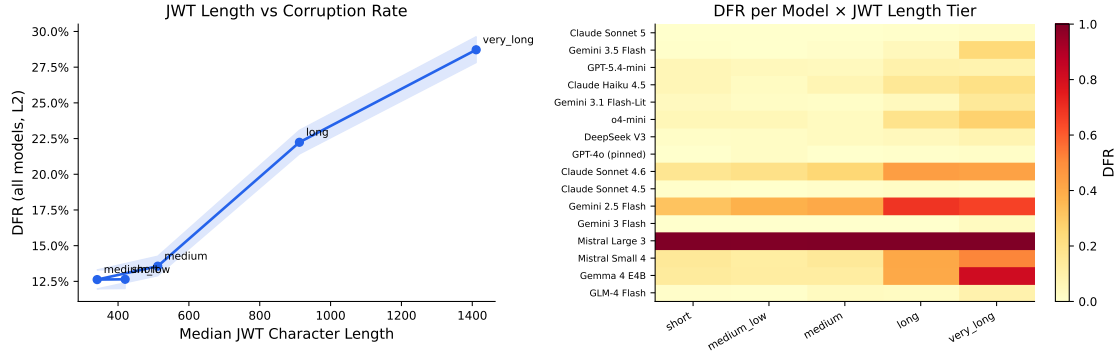


Fig. 2. DFR by JWT length tier and context depth (left) and per-model DFR heatmap across length tiers (right). The onset of non-trivial corruption occurs at ~ 200 characters.

tectural requirement. Character-level corruption (`corrupted`) requires a reference copy for verification. Semantic substitution (`sem_sub`), where the LLM decodes the JWT payload and returns a plausible but cryptographically invalid value, is undetectable by format validation, length checking, or bounded Levenshtein comparison: the only complete mitigation is to prevent the LLM from processing the credential at all. Model refusal (`refusal`) and abstention (`abstention`) mean the architecture cannot depend on the LLM forwarding the credential. The only mechanism that covers all five outcome categories is bypassing the LLM for credential relay entirely.

B. RA Type and Design Principles

Following Angelov et al. [27], we classify the Credential Interception Vault as a *Type 2 facilitation-type* reference architecture: designed to guide practitioners building privacy-preserving MCP agent systems, with voluntary adoption. The architecture is empirically grounded following Galster and Avgeriou’s [28] six steps: problem identification (Section I); evidence collection ($N=50,400$ runs); requirement derivation (six-outcome taxonomy, above); architecture proposal (this section); validation (eco arm and production, Section VI); and documentation (this paper and replication package).

The architecture rests on three design principles:

P1: Deterministic Shell, Probabilistic Core. The LLM (the probabilistic core) reasons about intent and selects tool invocations. The Vault (the deterministic shell) executes credential injection with correctness guarantees independent of the LLM’s inference quality. This embodies what Amodei et al. [33] identify as a necessary property of safe AI systems: deterministic oversight of probabilistic components. The Vault instantiates the Interceptor pattern from POSA [34] and is consistent with the decide/execute separation in Li et al.’s ACE architecture [24].

P2: Credentials by Reference, Not by Value. The LLM never receives the raw credential. It receives an opaque reference handle that carries no information about the credential’s content. The Vault resolves the handle to the cached credential at injection time.

P3: Model-Agnostic Guarantee. The correctness guarantee holds regardless of which LLM is in use. The Vault operates at

the MCP middleware layer, below the inference layer; swapping the underlying model requires no changes to the Vault.

C. Architecture Overview

Figure 3 presents the C4 container-level view [35].¹ The system boundary contains five containers. The *Credential Interception Vault* (centre, bold) is the architectural invariant: its intercept-cache-inject behaviour is fixed across all instantiations. All other containers are variability points ([VP]) that differ across deployments. The trust boundary (dashed) encloses the Vault and, in privacy-critical deployments, the Credential-Bearing Tool; credentials never cross this boundary to untrusted environments.

The five containers are the *Decision-Making Agent* (MCP client, uses only opaque handles); the *Credential Interception Vault* (intercepts, caches, and injects credentials); the optional *Tool Schema Aggregator*; the *Credential-Bearing Tool* [VP]; and the *Authenticated Tool* [VP]. External actors are the Service Consumer, LLM Inference Provider (receives only handles, never raw credentials), and Protected Resource [VP].

D. Intercept-Cache-Inject Sequence

Figure 4 shows the runtime behaviour. The Vault intercepts the credential-bearing tool result before it enters the LLM context (*intercept*), caches it under a fresh opaque handle (*cache*), and returns only the handle to the agent. When the agent issues a downstream tool call, the Vault resolves the handle and injects the credential into the tool arguments (*inject*). No LLM re-encoding occurs between cache and injection.

E. Security and Privacy Properties

Confidentiality. The credential never enters the LLM context window and is never transmitted to the LLM inference provider. In deployments where the provider is in a different jurisdiction and the credential constitutes personal data (e.g., a user-specific bearer token), this property aligns with GDPR Article 44 cross-border transfer restrictions [36].

Integrity. The cached credential is the authoritative copy. Downstream injection is byte-for-byte identical to the cached

¹The label *C4* here denotes the four-level container-diagram notation of Brown [35]; it is unrelated to the contribution identifier **C4** used in Sections I and IX.

Credential Interception Vault — Container Architecture

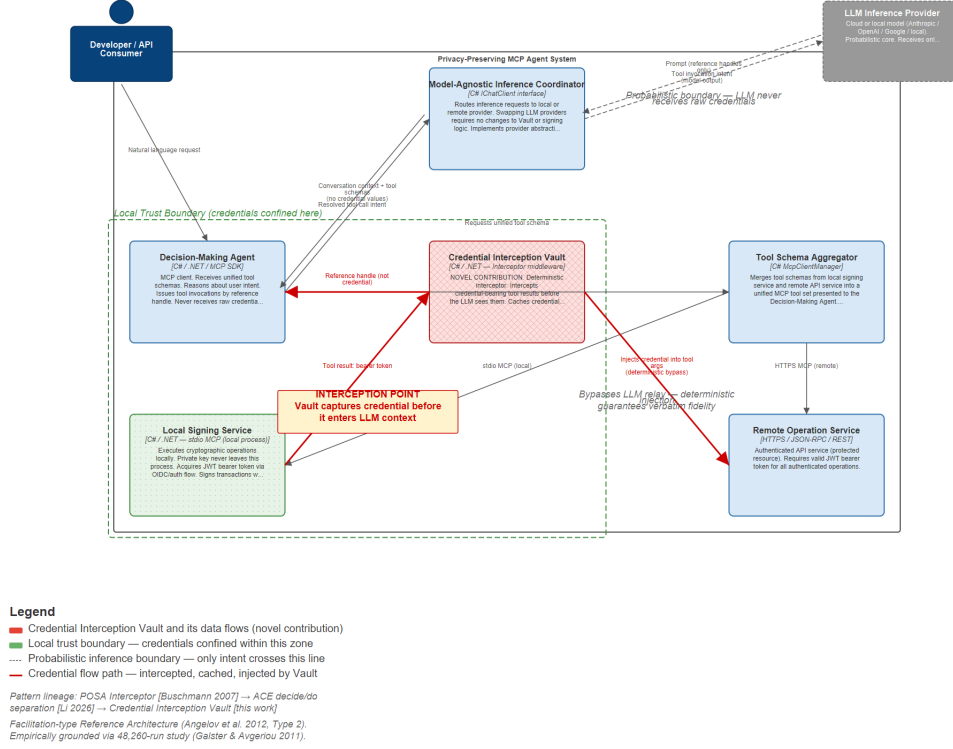


Fig. 3. Container-level view of the Credential Interception Vault reference architecture. The Vault (centre, bold red) is the invariant. Hatched elements are variability points [VP]. Handle security model: see Section V-E.

original: Levenshtein distance equals zero by construction, not by probabilistic suppression. Cryptographic signature verification at the Protected Resource succeeds with probability 1.0.

Availability and concurrency. The Vault runs in-process; network partitions cannot interrupt injection. Handles are session-scoped UUIDs, enabling stateless horizontal scaling.

Handle security model. Each handle is a cryptographically random session-scoped UUID that carries no information about the credential’s content; an attacker who obtains a handle cannot derive the credential from it. The handle lifecycle is: (i) *created* at intercept time and bound to the current session; (ii) *resolved* exactly once per downstream tool call; (iii) *expired* on session close or explicit invalidation, after which the handle resolves to null and injection fails safe (the tool call is rejected rather than forwarding a stale or empty credential). Cross-session replay is rejected by construction: a handle UUID is absent from a new session’s store and therefore cannot be resolved. The trust assumption is that the Vault runs within the local process trust boundary; credentials and handles never cross that boundary to untrusted environments.

Performance and interoperability. The intercept-inject cycle adds one in-memory lookup per relay event (sub-millisecond). The pattern is protocol-agnostic and applies to any framework interposing tool-call results before LLM context injection (LangChain, AutoGen, and similar runtimes); each hop in a multi-hop chain intercepts independently.

VI. INDUSTRIAL CASE STUDY

This section addresses **RQ5** using a mixed-method design: qualitative triangulation across five evidence sources is augmented by the ecological validity arm as an embedded controlled experiment (the no-Vault counterfactual).

A. Case Context

The case is a production blockchain trading platform operated by a Swiss-based fintech company serving approximately 13,000 registered users. The platform provides fixed-return options trading in tokenised financial instruments through an AI-native interface: an LLM orchestrator translates trader instructions into on-chain operations via MCP tool chains. The blockchain infrastructure is a private Proof-of-Authority (PoA) Ethereum-compatible network with four active nodes.

Authentication uses RS256 [5] JWTs [4] issued by Azure AD B2C at approximately 1,080 characters, placing the production tokens in the *very long* tier of our experimental design (Section III-A). A single character corruption invalidates the RSA signature and produces an HTTP 403 rejection at the downstream API [6]. The LLM configuration supports multiple providers (the controlled study includes Claude, GPT, and Gemini families); a local privacy-preserving mode using Gemma 4 E4B (4B-parameter open-weight, Ollama) is also supported for data-sovereignty deployments.

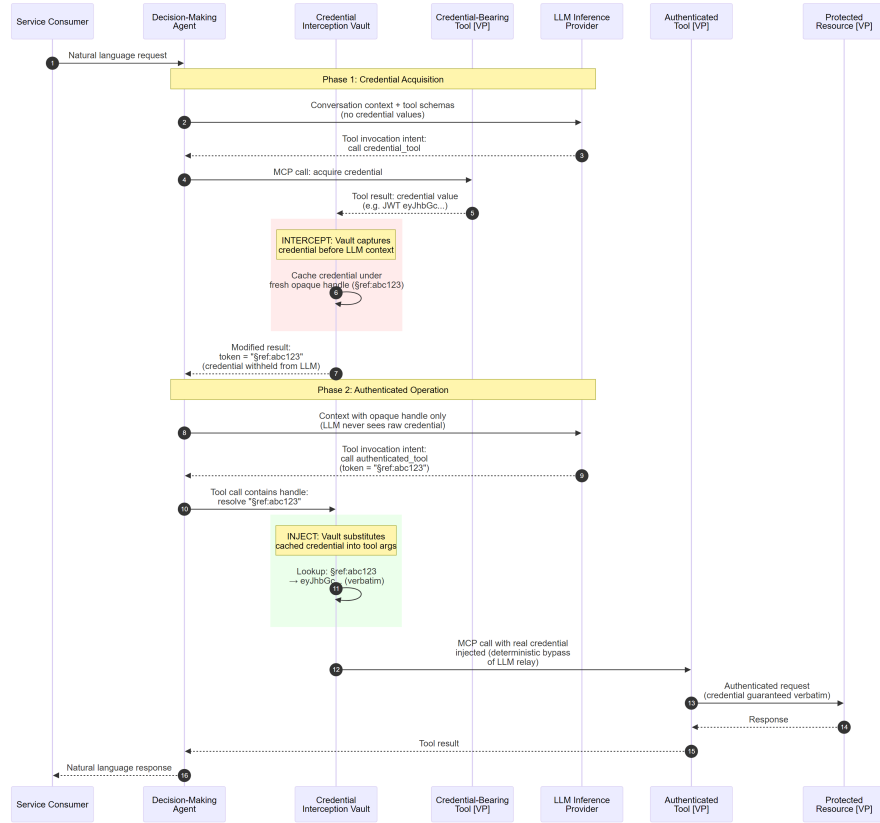


Fig. 4. Intercept-cache-inject sequence for one credential relay cycle. The credential never enters the LLM context window. Steps 5–6 (red): intercept and cache. Steps 10–12 (green): resolve and inject.

Concrete instantiation. [VP: LLM] = user-configurable model; [VP: Protected Resource] = PoA Ethereum RPC + REST trading API; [VP: Credential-Bearing Tool] = RS256 signing service issuing Azure AD B2C JWTs.

B. Evidence Sources

Five independent evidence sources are triangulated. (1) Production source code and git commit history provide an objective timeline of mitigations. (2) Contemporaneous developer communications document the misdiagnosis phase and the qualitative observation that prompting “gave better results.” (3) A structured interview with the company’s CEO and infrastructure engineer provides architectural context and co-author fact verification. (4) The ecological validity arm ($N = 1,600$) serves a dual role: it quantifies the production-specific DFR for the main study, and it constitutes the controlled counterfactual strand of this case study—the *no-Vault* condition instantiated on the actual production system via an environment variable flag (`RESEARCH_RAW_MODE=true`), providing a quantified baseline against which the post-Vault observation is interpreted. (5) Commit-level pre/post evidence confirms zero authentication failures after Vault deployment. No source contradicts another.

C. Discovery Trajectory

JWT corruption was first observed on 2026-04-03 during UAT testing, eight days after the MCP server was first

built. The observable symptom was intermittent HTTP 403 Unauthorized responses from the downstream REST API on authenticated trading operations. The intermittent failure rate was approximately 20%, consistent with the very-long tier DFR in the controlled study.

Misdiagnosis. The failure was attributed to infrastructure: signatures failed `RSA.VerifyData()` against all six active JWKS keys. One key was 47 days old, leading to the hypothesis that Azure AD was issuing tokens with a stale private key; a Microsoft Support case was raised. The misdiagnosis was plausible: a JWT corrupted by single-character substitution is cryptographically indistinguishable from a token signed with an invalid key.

Root cause identification. The lead developer, who had personally observed the corruption during testing, independently formed the LLM relay hypothesis from knowledge of how LLMs handle opaque, high-entropy strings. A diagnostic mechanism was added to the tool chain: the `acquire_bearer_token` tool was modified to return a `tokenFingerprint` (SHA256[:16] of the JWT), and a verification tool was added to compute the fingerprint of the received bearer token. Comparing fingerprints before and after LLM relay confirmed that the token was being altered in transit.

Prompting-first mitigation. The initial response followed a prompting-first trajectory: a 318-character CRITICAL verbatim fidelity instruction was added to the `bearerToken` parameter in the tool registry. The instruction explicitly names

the RSA signature consequence and the single-character precision requirement. The partial improvement confirmed the relay hypothesis but did not solve the problem. This is precisely the intervention evaluated in the ecological validity arm (Section VI-E): the same 318-character instruction, the same production JWT schema, producing 19.9% residual DFR.

Architectural solution. On 2026-04-13 (commit 5df1b265a), the lead developer deployed the Vault implementation, with the commit comment: “*Bearer token vault: bypass the LLM to prevent token corruption. The LLM hallucinates data into JWT tokens (expanding claims, appending fake responses). We intercept the token from acquire_bearer_token results and inject it directly.*” This comment identifies two failure modes observed in production: (1) semantic expansion, where the LLM hallucinated additional JSON claims into the JWT payload; and (2) content fabrication, where the LLM appended fabricated tool-response content after the JWT. Both are consistent with the failure taxonomy in Table IV.

The commit message uses no architectural pattern vocabulary; the solution was derived from first principles under production pressure. Pattern identification and naming were applied retrospectively, confirming the RA as a naturally emerging solution that this paper names, formalises, and validates.

D. Reactive Mitigations as Defence-in-Depth

Before the Vault was implemented, and retained alongside it as defence-in-depth, two reactive mitigation layers were added to the MCP handler. The first, `SanitizeBearerToken()`, handles structural failures: it extracts the `accessToken` field when the LLM passes the full JSON response object (case 1), strips non-Base64url characters appended after the signature (case 2), and removes Bearer prefix confusion (case 3). The second, `JwtValidator.ValidateAndRepairAsync()`, implements single-character brute-force repair of RS256 signatures: for each character position in the signature segment, it substitutes all 64 Base64url characters and tests `RSA.VerifyData()` against the JWKS keys. This repairs Levenshtein-1 corruption in the signature segment but cannot repair multi-character corruption or payload-level semantic substitution.

Table V maps these layers against the failure taxonomy. The reactive mitigations cover format errors and low-Levenshtein signature corruption but cannot detect semantic substitution (`sem_sub`), where the corrupted token is well-formed and plausible-length. The Vault is the only mechanism with complete coverage: it bypasses the failure mode entirely by removing the credential from the LLM’s inference path.

E. Ecological Validity

The eco arm ($N = 1,600$; 16 models $\times$ 100 iterations) is the controlled counterfactual for this case study. It used the exact production JWT schema (1,080-character RS256 Azure AD B2C token) and the verbatim 318-character

TABLE V
DEFENCE-IN-DEPTH COVERAGE MATRIX. REACTIVE MITIGATIONS COVER A SUBSET OF FAILURE MODES; THE VAULT COVERS ALL BY BYPASSING LLM RELAY.

Failure mode	Sanitize	Validator	Vault
JSON object relay	✓	Partial	✓
Trailing fabrication	✓	Partial	✓
Bearer prefix confusion	✓	Partial	✓
Char-level substitution	×	✓	✓
Multi-char corruption	×	×	✓
Abstention / refusal	×	×	✓

`BearerTokenParam` instruction from the production code-base, executed through the production software with Vault protection deliberately disabled. An environment variable flag (`RESEARCH_RAW_MODE=true`, commit 2396d5273, 2026-07-07) bypasses both `SanitizeBearerToken()` and `JwtValidator`, forwarding the raw LLM output verbatim to the downstream API. The production binary is identical in both modes; only the runtime environment variable differs, ensuring the treatment and control conditions share the same code path and differ solely in the presence or absence of Vault interception. This design was necessary because the production mitigations are effective enough to mask the underlying failure: without deliberately disabling them, the raw DFR is not observable.

The eco arm DFR was 19.9%, a +1.9 percentage point difference from the main study ($p = 0.16$, Wilcoxon; not significant). The Spearman rank correlation between eco arm and main experiment model rankings was $\rho = 0.740$ ($p = 0.001$), confirming that model rankings transfer to production conditions.

F. Post-Vault Deployment

Zero JWT authentication failures have been recorded over the 88-day observation window from Vault deployment (2026-04-13) through 2026-07-10. Azure Monitor metrics record 4,774 MCP tool invocations during this window; at the eco arm’s 19.9% DFR, approximately 950 would have failed without Vault protection. The eco arm (Section VI-E) provides the quantified counterfactual on the same production system. The post-Vault zero-failure observation is interpreted against that baseline, not in isolation. This is a structural result, not a statistical one: the Vault eliminates the failure mode by removing the credential from the LLM’s inference path. The expected post-Vault DFR is not “lower than 18.0%” but exactly zero by construction for the byte-relay property: the byte sequence injected into each outbound request is identical to the credential stored in the Vault. This guarantee does not extend to credential theft, token expiry, revocation, or replay attacks, which require independent controls.

VII. DISCUSSION

The empirical findings and case study evidence converge on a single conclusion: LLM credential relay corruption is a systematic failure mode that prompt engineering did not eliminate in any condition tested, motivating an architectural response. We discuss six implications.

Prompt engineering as insufficient mitigation. The production system deployed the most explicit bearerToken instruction achievable within the MCP tool schema (318 characters naming the RSA mechanism, the 403 consequence, and single-character precision). It produced 19.9% DFR ($p = 0.16$; a null result, not a positive equivalence claim). BPE tokenisation boundaries *appear* to operate below the instruction-following layer; no instruction eliminated corruption in any of the 16 models tested. In the conditions evaluated, prompt improvement was bounded at ~ 5 pp (L1-to-L3) with no approach toward zero. We present this as an *observed practical insufficiency*: the most aggressive prompting strategy available under MCP schema constraints did not reduce DFR to an acceptable level for cryptographic relay. Formal equivalence testing (TOST with a pre-specified margin) remains future work; the current finding is sufficient to motivate an architectural bypass.

Mitigation vs. bypass: a category difference. The Vault achieves zero DFR by construction for the byte-relay property (a design argument, not an additional empirical measurement): removing the credential from inference makes byte-level corruption architecturally impossible. This guarantee is scoped to relay fidelity; it does not address credential theft at rest, token expiry, revocation, or replay attacks; those remain the responsibility of the surrounding identity infrastructure. Prompt-based mitigation asks how to make the LLM relay more reliably; DFR remains above zero. These are solutions to different problems. Bypass is appropriate when correctness is required (RSA verification is binary), when the failure is unrecoverable (HTTP 403 with no retry), and when credentials are sensitive personal data [36].

When to apply the Vault. Apply when: (i) credential length exceeds ~ 200 chars; (ii) a cryptographic signature is present; or (iii) a relay error is unrecoverable. Prompt mitigation may suffice in retry-tolerant contexts below that threshold.

Organic convergence validates the RA. The production Vault was implemented independently without reference to this research (Section VI-C), confirming the RA as naturally emergent and transferable without requiring rediscovery.

Credential lengths are increasing; the risk may worsen. Gibberish bias intensifies with larger vocabularies [13]. NIST key sizes [37] and post-quantum algorithms [38] (CRYSTALS-Dilithium: 2,420 bytes; SPHINCS+: 7,856 bytes) would push tokens deeper into the high-DFR tier, making 17.3% a conservative lower bound (this projection requires empirical validation). The Vault addresses this trajectory without modification.

Broader implications for agentic AI systems. As LLM-based agent systems proliferate [39], any opaque high-entropy string (API keys, hash digests, certificate fragments) may share the same distribution-shift vulnerability. JWT relay is the validated case; extension to other credential classes is a hypothesis requiring independent empirical confirmation. The eco arm ($\rho = 0.740$) provides a reusable calibration instrument for deployment-specific risk.

VIII. THREATS TO VALIDITY

Construct validity. The binary DFR metric does not capture corruption severity, but RSA verification is itself binary (any distance > 0 is fatal), making the metric appropriate. Synthetic JWTs may under-represent real-world complexity; the eco arm mitigates this by running with the exact production schema ($\sim 1,080$ chars, RS256 Azure AD B2C) and obtaining DFR = 19.9%, statistically indistinguishable from the synthetic result.

Internal validity. Experiments used direct provider API calls; identical corruption patterns across six independent provider SDKs make a middleware artifact implausible. All models were accessed under pinned version identifiers during a fixed collection window of 3–7 July 2026; any residual within-study drift is accounted for by the GEE clustering structure. Provider back-end updates between study start and end could introduce uncontrolled variance; the fixed collection window and version pinning limit but do not eliminate this risk. The tokenisation hypothesis is the most parsimonious explanation for the observed length-gating pattern; alternative mechanisms (tool-call serialisation artefacts, provider SDK transforms, safety-layer rewrites) have not been experimentally ruled out. Causal attribution is out of scope for this study, whose primary contribution is demonstrating that the problem exists at meaningful scale and that an architectural bypass eliminates it.

External validity. The 16 models cover eight tiers and six providers but do not represent the full model landscape; new models may exhibit different failure profiles. The eco arm’s $\rho = 0.740$ rank correlation supports transfer to at least one production deployment; generalisation to other credential types or schemas requires future work (Section VII). Per-model eco breakdowns are in the replication package.

Conclusion validity. Multiple comparisons across 16 models inflate Type I error; all pairwise comparisons are corrected by Benjamini-Hochberg at FDR = 0.05. The primary inference (DFR $> 0\%$ across all models) is a single test with effectively unlimited power at $N = 50,400$. The analysis plan was pre-registered before data collection to reduce observer bias.

IX. CONCLUSION

LLM credential relay corruption is not an edge case. Across 50,400 main-arm runs with 16 frontier models, the JWT defect rate was 18.0% (range 0.5–99.8%; 12.5% excluding Mistral Large 3), with no model achieving zero corruption. The eco arm confirmed a residual DFR of 19.9% under the most explicit production instruction ($p = 0.16$). The Credential Interception Vault eliminates the failure by removing the credential from inference; zero authentication failures have been recorded since deployment.

The seven contributions (Section I) span empirical characterisation (C1–C4), the Credential Interception Vault RA (C5), industrial validation (C6), and an open replication package (C7).

Credential lengths are increasing [37], [38], deepening the corruption risk. The Vault provides a principled, model-

agnostic, and empirically validated answer. For practitioners building MCP tool chains: do not relay credentials exceeding ~200 characters through LLM inference; intercept them deterministically at the middleware layer.

ACKNOWLEDGMENT

This research was conducted within [anonymised], the [anonymised] Research Centre for Software, funded by [anonymised] under grant [anonymised] in conjunction with [anonymised] under statement of work [anonymised].

ETHICS STATEMENT

Ethical approval was granted by [institution withheld] under expedited review (reference withheld). The industry partner provided written consent for the case study.

AI DISCLOSURE

Microsoft Copilot assisted with spelling and grammar; all intellectual content is the authors' own.

DATA AVAILABILITY

Replication package: <https://anonymous.4open.science/r/jwt-mcp-replication-FF50/>. Experiment database (~161 MB, 52,000 runs): <https://osf.io/5zd42/> (SHA-256: `ac1de7ac...fdc5c0e`).

REFERENCES

- [1] Anthropic, “Model context protocol specification,” <https://modelcontextprotocol.io/specification/2025-06-18>, 2024, technical specification.
- [2] H. Guo, Y. Hao, Y. Zhang, M. Xu, P. Lv, J. Chen, and X. Cheng, “A measurement study of model context protocol ecosystem,” 2025.
- [3] National Security Agency, “Model context protocol (MCP): Security design considerations for AI-driven automation,” National Security Agency, Tech. Rep. U/OO/6030316-26, May 2026. [Online]. Available: https://www.nsa.gov/Portals/75/documents/Cybersecurity/CSL_MCP_SECURITY.pdf
- [4] M. B. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” RFC Editor, RFC 7519, May 2015. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc7519>
- [5] M. B. Jones, “JSON Web Algorithms (JWA),” RFC Editor, RFC 7518, May 2015. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc7518>
- [6] M. B. Jones and D. Hardt, “The OAuth 2.0 Authorization Framework: Bearer Token Usage,” RFC Editor, RFC 6750, October 2012. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc6750>
- [7] [Anonymous], “[title withheld for double-anonymous review],” in *Proc. IEEE Int. Symp. on Software Reliability Engineering (ISSRE)*, 2026, details withheld for review.
- [8] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin, and T. Liu, “A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions,” *ACM Transactions on Information Systems*, 2025.
- [9] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, “Survey of hallucination in natural language generation,” *ACM Computing Surveys*, vol. 55, no. 12, 2023.
- [10] Y. Nie *et al.*, “Decoding secret memorisation in code LLMs through token-level characterisation,” in *Proc. IEEE/ACM Int. Conf. on Software Engineering (ICSE)*, 2025.
- [11] P. Abadeh, M. Lochner, T. Ansari, and F. Zarrinkalam, “MCP-Scanner: Detecting security risks in model context protocol systems,” in *Proc. 2026 ACM/IEEE 7th Int. Workshop on Engineering and Cybersecurity of Critical Systems (EnCyCriS)*, ser. EnCyCriS '26. Association for Computing Machinery, 2026, pp. 41–48.
- [12] R. Sennrich, B. Haddow, and A. Birch, “Neural machine translation of rare words with subword units,” in *Proc. 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2016.
- [13] M. Chen *et al.*, “Understanding secret leakage risks in code LLMs: A tokenization perspective,” in *Findings of the Association for Computational Linguistics (ACL)*, 2026.
- [14] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, “Toolformer: Language models can teach themselves to use tools,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [15] F. He, T. Zhu, D. Ye, B. Liu, W. Zhou, and P. S. Yu, “The emerged security and privacy of LLM agent: A survey with case studies,” *ACM Computing Surveys*, vol. 58, no. 6, pp. 1–36, Dec. 2025.
- [16] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection,” in *Proc. Workshop on Artificial Intelligence and Security (AISeC@CCS)*, 2023.
- [17] Y. Liu *et al.*, “Formalizing and benchmarking prompt injection attacks and defenses,” in *Proc. USENIX Security Symposium*, 2024.
- [18] Y. Song *et al.*, “CallNavi: A challenge and empirical study on LLM function calling and routing,” in *Proc. 29th Int. Conf. on Evaluation and Assessment in Software Engineering (EASE)*, 2025.
- [19] P. Yu *et al.*, “Benchmarking LLM tool-use in the wild,” in *Proc. 14th Int. Conf. on Learning Representations (ICLR)*, 2026. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2026/hash/4fb6b4482e4e24a33a3de5b01a011eb0-Abstract-Conference.html
- [20] M. A. Haque *et al.*, “Verbatim data transcription failures in LLM code generation: A state-tracking stress test,” 2026. [Online]. Available: <https://arxiv.org/abs/2601.03640>
- [21] T. Yarygina and A. H. Bagge, “Overcoming security challenges in microservice architectures,” in *Proc. 12th IEEE Symp. on Service-Oriented System Engineering (SOSE)*, 2018.
- [22] M. Shamsujjoha, Q. Lu, D. Zhao, and L. Zhu, “Swiss cheese model for AI safety: A taxonomy and reference architecture for multi-layered guardrails of foundation model based agents,” in *Proc. IEEE 22nd Int. Conf. on Software Architecture (ICSA)*, 2025, pp. 37–48.
- [23] J. Reason, *Human Error*. Cambridge University Press, 1990.
- [24] E. Li *et al.*, “ACE: A security architecture for LLM-integrated app systems,” in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2026.
- [25] S. Geng, M. Josifoski, M. Peyrard, and R. West, “Grammar-constrained decoding for structured NLP tasks without finetuning,” in *Proc. Conf. on Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 10932–10952.
- [26] M. Jones, A. Nadalin, B. Campbell, J. Bradley, and C. Mortimore, “OAuth 2.0 token exchange,” IETF, Tech. Rep. RFC 8693, 2020. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc8693>
- [27] S. Angelov, P. Grefen, and D. Greefhorst, “A framework for analysis and design of software reference architectures,” *Information and Software Technology*, vol. 54, no. 4, pp. 417–431, 2012.
- [28] M. Galster and P. Avgeriou, “Empirically-grounded reference architectures: a proposal,” in *Proc. Joint ACM SIGSOFT Conf. – QoSA and ACM SIGSOFT Symp. – ISARCS*, 2011, pp. 153–158.
- [29] F. Shull, J. Singer, and D. I. K. Sjøberg, *Guide to Advanced Empirical Software Engineering*. Springer, 2008.
- [30] P. Runeson and M. Höst, “Guidelines for conducting and reporting case study research in software engineering,” *Empirical Software Engineering*, vol. 14, no. 2, pp. 131–164, 2009.
- [31] R. Verdecchia and J. Bogner, “Notes on writing effective empirical software engineering papers,” in *Proc. IEEE/ACM Int. Conf. on Software Engineering (ICSE), Companion Volume*, 2025.
- [32] B. A. Kitchenham, S. L. Pflieger, L. M. Pickard, P. W. Jones, D. C. Hoaglin, K. El Emam, and J. Rosenberg, “Preliminary guidelines for empirical research in software engineering,” *IEEE Transactions on Software Engineering*, vol. 28, no. 8, pp. 721–734, Aug. 2002.
- [33] D. Amodi, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané, “Concrete problems in AI safety,” *arXiv preprint arXiv:1606.06565*, 2016.
- [34] F. Buschmann, K. Henney, and D. C. Schmidt, *Pattern-Oriented Software Architecture, Volume 4: A Pattern Language for Distributed Computing*. John Wiley & Sons, 2007.
- [35] S. Brown, *Software Architecture for Developers, Volume 2: Visualise, document and explore your software architecture*. LeanPub, 2018, available: <https://leanpub.com/visualising-software-architecture>.
- [36] European Parliament and Council of the European Union, “Regulation (EU) 2016/679 of the european parliament and of the council on the protection of natural persons with regard to the processing of personal data and on the free movement of such data (General Data Protection Regulation),” Official Journal of the European Union, L 119, 1–88, May 2016. [Online]. Available: <https://eur-lex.europa.eu/eli/reg/2016/679/oj>
- [37] E. Barker, “Recommendation for key management: Part 1 – general,” National Institute of Standards and Technology, NIST Special Publication 800-57 Part 1 Rev. 5, 2020.

- [38] National Institute of Standards and Technology, “Module-lattice-based digital signature standard (FIPS 204), stateless hash-based digital signature standard (FIPS 205), and SLH-DSA (FIPS 206),” NIST, Tech. Rep., 2024, post-Quantum Cryptography Standardization.
- [39] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J. Wen, “A survey on large language model based autonomous agents,” *Frontiers of Computer Science*, vol. 18, no. 6, p. 186345, 2024.